ADP470S

Data Structures & Algorithms

Term 1 — Complete Study Notes

Includes: Quick Sort | Insertion Sort | Recursion vs Iteration Big-O

Cape Peninsula University of Technology | 2026

SO1 — Basic Introduction to Algorithms & Complexity

SO1 — Algorithms & Complexity

What is an Algorithm?

An **algorithm** is a finite, ordered set of well-defined instructions for solving a problem. Every algorithm must have:

- **Input:** zero or more values are provided
- **Output:** at least one result is produced
- **Definiteness:** every step is clear and unambiguous
- **Finiteness:** it must stop after a finite number of steps
- **Effectiveness:** every step can actually be carried out

Big-O Notation

Big-O describes the **upper bound** of how an algorithm's resource usage grows as input size **n** grows. Always drop constants and lower-order terms.

Notation	Name	Example	n=1000 steps
O(1)	Constant	Array index: arr[5]	1
O(log n)	Logarithmic	Binary Search	~10
O(n)	Linear	Single loop, Linear Search	1,000
O(n log n)	Linearithmic	Quick Sort (avg), Merge Sort	~10,000
O(n^2)	Quadratic	Nested loops, Bubble Sort	1,000,000
O(2^n)	Exponential	Naive recursive Fibonacci	Astronomical

Big-O simplification rules:

- Drop constants: O(3n) = O(n)

- Drop lower terms: O(n^2 + n) = O(n^2)
- Sequential loops ADD: O(n) + O(n) = O(n)
- Nested loops MULTIPLY: O(n) x O(n) = O(n^2)
- Loop variable that multiplies (j = j*2 or j*3) gives O(log n), not O(n)

SO2 — Recursion vs Iteration — Factorial & Fibonacci

SO2 — Recursion vs Iteration

Iteration

Uses a **loop** to repeat steps. Only one stack frame ever used in memory.

Iterative Factorial:

```
public static int factorial(int n) {
    int prod = 1;
    for (int i = 1; i <= n; i++) {
        prod *= i;
    }
    return prod;
}
```

Metric	Iterative Factorial
Time	O(n) — loop runs n times
Space	O(1) — only one variable 'prod', never grows

Recursion

A method that **calls itself** with a smaller input. Must have a **base case** to stop. Each call creates a new **stack frame** in memory.

Recursive Factorial:

```
public static int fact(int n) {
    if (n == 0) return 1;         // BASE CASE — stops the recursion
    return n * fact(n - 1);       // RECURSIVE CASE — smaller input
}
```

Call stack for fact(4) — ALL frames alive at the same time:

Stack (top = newest)	Returns
fact(0) <-- BASE CASE	1
fact(1) = 1 x fact(0)	1
fact(2) = 2 x fact(1)	2
fact(3) = 3 x fact(2)	6

fact(4) = 4 x fact(3) <-- first call	24

Metric	Recursive Factorial
Time	O(n) — one call per decrement of n
Space	O(n) — n+1 stack frames alive simultaneously on the call stack

Remember: Every recursive method MUST have a base case. Without it the method never stops — StackOverflowError!

Recursion vs Iteration — Big-O Comparison

Property	Iteration	Recursion
Time complexity	O(n)	O(n)
Space complexity	O(1) — constant, just loop variables	O(n) — one stack frame per call, all alive at once
Stack overflow risk	None	Yes — large n will crash the program
Function-call overhead	None	Yes — one overhead per recursive call
Readability	More verbose code	Cleaner, mirrors mathematical definition
Real-world preference	Preferred for factorial / Fibonacci	Trees, graphs, divide-and-conquer problems

Key point: Both recursion and iteration are O(n) in TIME for factorial. The critical difference is SPACE: iteration = O(1), recursion = O(n).

Fibonacci

Sequence: F(0)=1, F(1)=1, F(n) = F(n-1) + F(n-2) → 1, 1, 2, 3, 5, 8, 13, 21 ...

Iterative Fibonacci — O(n) time, O(1) space:

```
public static void fibonacciViaIteration(int n) {
    int first = 1, second = 1;
    System.out.println(first);
    System.out.println(second);
    for (int i = 2; i <= n; i++) {
        int third = first + second;
        System.out.println(third);
        first = second;
        second = third;
    }
}
```

Recursive Fibonacci — O(2^n) time, O(n) space:

```
public static int fib(int n) {
    if (n == 0 || n == 1) return 1;    // base case
    return fib(n - 1) + fib(n - 2);    // TWO recursive calls — causes
explosion
```

```
}
```

Method	Time	Space	Why
Iterative factorial	O(n)	O(1)	One loop, one variable
Recursive factorial	O(n)	O(n)	n+1 stack frames all in memory at once
Iterative Fibonacci	O(n)	O(1)	One loop, two variables
Recursive Fibonacci	O(2^n)	O(n)	Two calls per step — doubles every level. Recalculates same values repeatedly

> **Remember:** Recursive Fibonacci makes over 2 billion calls for n=50. Always use the iterative version in practice.

SO3 — Complexity — Linear Search & Binary Search

SO3 — Search Algorithm Complexity

Linear Search — O(n)

Check each element **one by one** from start to end until target is found.

```
public static int linearSearch(int[] numbers, int target) {
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] == target) return i;
    }
    return -1;
}
```

Case	Complexity	When
Best	O(1)	Target is first element
Average	O(n)	Target is in the middle
Worst	O(n)	Target is last or not present

Binary Search — O(log n)

Requirement: array MUST be sorted. Repeatedly halves the search space.

```
public static int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target)      return mid;
        else if (arr[mid] < target)  low = mid + 1;
        else                         high = mid - 1;
    }
    return -1;
}
```

Case	Complexity	When
Best	O(1)	Target is at the middle first try
Average	O(log n)	Target found after several halvings
Worst	O(log n)	Target at extreme end or not present

log2(1024) = **10 steps**. Linear search = up to 1024 steps. Binary search = only 10.

SO4 — Bubble Sort & Best/Worst/Average Case Complexity

SO4 — Bubble Sort

How It Works

Compare **adjacent pairs**. Swap if left > right. After each pass, the largest unsorted element 'bubbles' to its correct position at the right end.

Java implementation:

```
public static void bubbleSort(int[] numbers) {
    int n = numbers.length;
    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (numbers[j] > numbers[j + 1]) {
                int temp = numbers[j];
                numbers[j] = numbers[j + 1];
                numbers[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

Full Trace — {90, 23, 5, 109, 12}

Pass	Array after pass	Element settled
Pass 1	{23, 5, 90, 12, 109}	109 at position [4]
Pass 2	{5, 23, 12, 90, 109}	90 at position [3]
Pass 3	{5, 12, 23, 90, 109}	23 at position [2]
Pass 4	{5, 12, 23, 90, 109}	No swaps — early exit — DONE

Final sorted: {5, 12, 23, 90, 109}

Complexity

Case	Complexity	Input	Why
Best	O(n)	Already sorted	One pass, zero swaps. Early-exit flag detects and stops.
Average	O(n^2)	Random order	Two nested loops ~n times each.
Worst	O(n^2)	Reverse sorted	Every element swaps all the way every pass.

Metric	Value
Time (avg/worst)	O(n^2)
Time (best — with early-exit flag)	O(n)
Space	O(1) — sorts in-place, no extra array

SO5 — 2D Arrays & Introduction to Linked Lists

SO5 — Arrays & Introduction to Linked Lists

2D Arrays

An **array of arrays**. Access any element using [row][column].

```
int[][] grid = { {1,2,3}, {4,5,6}, {7,8,9} };
System.out.println(grid[1][2]);  // 6  (row 1, col 2)
```

```
for (int i = 0; i < grid.length; i++) {
    for (int j = 0; j < grid[i].length; j++) {
        System.out.print(grid[i][j] + " ");
    }
    System.out.println();
}
```

Key point: Traversing an n x n 2D array = O(n^2). For n rows x m columns = O(n x m).

Limitations of Arrays

Limitation	Problem	Cost
Fixed size	Size declared upfront. Cannot grow.	Wasted memory or overflow
Slow insertion	Shifts all elements right.	O(n) per insert
Slow deletion	Shifts all elements left.	O(n) per delete
Contiguous memory	Must fit in one block.	Hard for large arrays

Introduction to Linked Lists

A chain of **nodes**. Each node stores: (1) data, (2) a pointer to the next node.

HEAD → [10|next] → [20|next] → [30|next] → NULL

```
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; this.next = null; }
}
```

Array vs Linked List — Complexity

Operation	Array	Linked List
Access by index	O(1)	O(n)
Insert at beginning	O(n)	O(1)
Insert at end	O(1) if space	O(n)
Delete element	O(n)	O(1) once found
Size	Fixed	Dynamic

SO6 — Doubly & Circular Linked Lists

SO6 — Doubly & Circular Linked Lists

Limitations of Singly Linked List

Limitation	Problem
One direction only	Can only move forward, never backward.
Deletion needs predecessor	Can't go back to find the previous node.
No tail reference	Must traverse full list to reach the end = O(n).

Doubly Linked List

Each node has **next** (forward) AND **prev** (backward) pointers.

NULL ← [prev|10|next] ⇄ [prev|20|next] ⇄ [prev|30|next] → NULL

```
class DNode {
    int data;
    DNode next;
    DNode prev;
    DNode(int data) { this.data = data; this.next = null; this.prev =
null; }
}
```

- Traverse both directions
- Easier deletion — previous node already known
- Use case: browser back/forward history, undo/redo

Remember: Trade-off: more memory. Two pointers per node.

Circular Linked List

Last node's **next** points back to **HEAD** instead of NULL.

HEAD → [10|next] → [20|next] → [30|next] → back to HEAD

- Music playlist on repeat
- Round-robin CPU scheduling
- Multiplayer game turns

All Linked List Types

Type	Pointers	End points to	Direction	Use case
Singly	1 (next)	NULL	Forward only	Stacks, queues
Doubly	2 (next+prev)	NULL	Both	Browser history
Singly Circular	1 (next)	HEAD	Forward, loops	Playlists
Doubly Circular	2 (next+prev)	HEAD	Both, loops	Complex navigation

---

NEW — Insertion Sort

Insertion Sort

How It Works

Builds a **sorted region** from left to right. For each new element (the **key**), shift all larger elements in the sorted region one position right, then insert the key into the gap.

Java implementation:

```
public static void insertionSort(int[] arr) {
    int n = arr.length;
    for (int i = 1; i < n; i++) {
        int key = arr[i];              // element to insert
        int j = i - 1;
        // shift larger elements right
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;              // place key in correct spot
    }
}
```

Full Trace — {90, 23, 5, 109, 12}

Starting state: Sorted = [90] | Unsorted = [23, 5, 109, 12]

Step	Key	Action	Array after
Start	—	First element (90) is trivially sorted	{90 | 23, 5, 109, 12}
1	23	23 < 90: shift 90 right. Insert 23 before it.	{23, 90 | 5, 109, 12}
2	5	5 < 90: shift. 5 < 23: shift. Insert 5 at front.	{5, 23, 90 | 109, 12}
3	109	109 > 90: no shifts. Insert 109 after 90.	{5, 23, 90, 109 | 12}
4	12	12 < 109,90,23: shift all three. 12 > 5: insert.	{5, 12, 23, 90, 109}

Final sorted: {5, 12, 23, 90, 109}

Complexity

Case	Complexity	When	Why
Best	O(n)	Already sorted	Inner while loop never executes. Just n comparisons.
Average	O(n^2)	Random order	On average n/2 shifts per element = n x n/2 = O(n^2).
Worst	O(n^2)	Reverse sorted	Every element shifts all the way to the front every step.

Metric	Value
Time (best)	O(n)
Time (avg/worst)	O(n^2)
Space	O(1) — sorts in-place

Bubble Sort vs Insertion Sort

Property	Bubble Sort	Insertion Sort
Mechanism	Swap adjacent pairs each pass	Shift elements right, insert key
Best case	O(n) — needs early-exit flag	O(n) — naturally if already sorted
Worst case	O(n^2)	O(n^2)
Practical speed	Slower — more swaps	Faster — fewer total operations
Space	O(1)	O(1)
Best for	Teaching / demonstration	Nearly sorted real-world data

---

NEW — Quick Sort

Quick Sort

How It Works

Choose a **pivot** element. **Partition**: move all elements less than pivot to the left, all greater to the right. The pivot is now in its **final correct position**. Recursively sort each side.

- **Step 1 — Choose pivot:** commonly the middle element, first, last, or random
- **Step 2 — Partition:** rearrange so everything < pivot is left, > pivot is right
- **Step 3 — Recurse:** apply Quick Sort to both left and right subarrays
- **Base case:** subarray of size 0 or 1 is already sorted — stop

Java implementation:

```
public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {                          // base case: 1 or 0
elements
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);  // sort left side
        quickSort(arr, pivotIndex + 1, high); // sort right side
    }
}

public static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];   // using last element as pivot
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
    }
    int temp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = temp;
    return i + 1;   // pivot is now at correct position
}
```

Full Trace — {90, 23, 5, 109, 12} (middle element as pivot)

Pivot index rule: floor((low + high) / 2)

Step 1 — Full array [0..4], pivot = index 2 = value 5

Array: {90, 23, **5**, 109, 12}

Region	Elements
Left (< 5)	{ } empty
Pivot	5
Right (> 5)	{90, 23, 109, 12}

Step 2 — Sort right {90,23,109,12}, pivot = index 1 = value 23

Array: {90, **23**, 109, 12}

Region	Elements
Left (< 23)	{ } empty
Pivot	23

Right (> 23)	{90, 109, 12}

Step 3 — Sort {90,109,12}, pivot = index 1 = value 109

Array: {90, **109**, 12}

Region	Elements
Left (< 109)	{90, 12}
Pivot	109
Right (> 109)	{ } empty

Step 4 — Sort {90,12}, pivot = index 0 = value 90

Array: {**90**, 12}

Region	Elements
Left (< 90)	{12}
Pivot	90
Right (> 90)	{ } empty

All subarrays are size 1 or empty — done.

Final sorted: {5, 12, 23, 90, 109}

Complexity

Case	Complexity	When it occurs	Why
Best	O(n log n)	Pivot always splits array into equal halves	log n recursion levels x O(n) work per level for partitioning.
Average	O(n log n)	Random input — typical real-world data	Near-balanced splits on average. About 1.39n log n comparisons.
Worst	O(n^2)	Pivot is always min or max element	n levels of unbalanced recursion x O(n) work each. Happens on sorted arrays with first/last pivot.

Metric	Value
Time (best/avg)	O(n log n)
Time (worst)	O(n^2)
Space	O(log n) — recursive call stack

> **Tip:** Choosing the middle element as pivot greatly reduces the chance of O(n^2) worst case compared to always picking first or last.

Quick Sort vs Bubble Sort vs Insertion Sort

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No

Key point: Quick Sort is much faster in practice than Bubble and Insertion Sort for large datasets, even though worst case is the same O(n^2).

Master Reference — All Algorithm Big-O

Algorithm / Pattern	Best	Average	Worst	Space
Array index access	O(1)	O(1)	O(1)	O(1)
Single loop	O(n)	O(n)	O(n)	O(1)
Two sequential loops	O(n)	O(n)	O(n)	O(1)
Nested loops (j++)	O(n^2)	O(n^2)	O(n^2)	O(1)
Nested loop (j=j*2/3)	O(n log n)	O(n log n)	O(n log n)	O(1)
Iterative factorial	O(n)	O(n)	O(n)	O(1)
Recursive factorial	O(n)	O(n)	O(n)	O(n)
Iterative Fibonacci	O(n)	O(n)	O(n)	O(1)
Recursive Fibonacci	O(2^n)	O(2^n)	O(2^n)	O(n)
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search	O(1)	O(log n)	O(log n)	O(1)
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)

ADP470S 2026 — Good luck!